6. Generators and Iterators

Understanding how generators work in Python.

➢ What is a Generator?

- A generator is a function that behaves like an iterator. Instead of returning all values at once, it yields one value at a time using the yield keyword.
- a generator is a special type of function that returns an iterator and allows you to generate values one at a time, using the yield keyword.

Difference between yield and return.

➢ Yield

- Used in: Generator functions.
- Returns: A generator object, which can be iterated over.
- Multiple values: Can be used to yield multiple values one at a time.

➢ Return

- Used in: Regular functions.
- Returns: A single value (or a tuple, list, etc.).
- One-time return: Can only return once per call.

Understanding iterators and creating custom iterators.

➢ What is an Iterator?

- An iterator is an object in Python that allows you to traverse through a collection, one element at a time, without needing to understand the underlying structure.
- The __iter__() method

➢ Custom Iterators

- You create a custom iterator when you want to:
- Control iteration behavior.
- Generate a sequence lazily (one value at a time).
- Create infinite or complex sequences (e.g., Fibonacci, prime numbers, etc.).

➢ Steps to Create a Custom Iterator

1. Define a class.
2. Implement __iter__(self) method → Returns the iterator object (often self).
3. Implement __next__(self) method → Computes and returns the next item, or raises StopIteration when done.

➢ When to Use Iterators

- When processing large data streams.
- When implementing complex looping logic.
- When you want memory-efficient iteration (lazy evaluation).